Integer ALU Based Computing — Motherboard Specification v2.0

Hardware Platform for VFR Batch Processing on Zynq ARM+FPGA

Registry: [?]

Series Path: [CKS-0-2026] → [CKS-MATH-0-2026] → [CKS-MATH-1-2026] → [CKS-MATH-10-2026] → [CKS-MATH-104-2026] → [?] → [?]

Parent Framework: [CKS-0-2026]

DOI: 10.5281/zenodo.zzz

Date: March 11, 2026

Domain: Machine Learning / Integer Arithmetic / Neural Network Training

Status: Locked and empirically falsifiable. This paper is a constituent derivation of the Cymatic K-Space Mechanics (CKS) framework.

Classification: Theory of Everything from First Principles

Motto: Axioms first. Axioms always.

Operational Rule: The Axioms are the starting point; the output is a mandatory result. Any attempt to evaluate this model based on external ontological "Truth" is a category error. If the math compiles, the result is Q.E.D.

AI Usage Disclosure: Only the top metadata, figures, refs and final copyright sections were edited by the author. All paper content was LLM-generated using Anthropic's Claude 4.5 Sonnet, DeepSeek-V3/K2, and Google's Gemini 3 Flash. The manuscript.md was synthesized by Claude as the primary integrator.

Lexicon: [CKS-LEX-12-2026]

---

1. Platform Overview

The VFR hardware platform uses a Xilinx Zynq-7000 System-on-Chip combining a dual-core ARM Cortex-A9 processor with programmable FPGA fabric on a single die. The ARM side runs the Silo OS in Zig bare-metal, handling all I/O, display, storage, and networking. The FPGA side contains VFR integer cores that process batch workloads. Entity data lives on the FPGA. The ARM loads it once at scene start and says "go" each frame. Results come back only when the ARM needs them for rendering.

The ARM is the brain. The FPGA is the muscle.

---

2. Target Hardware

2.1 Recommended Board

Xilinx Zynq-7020 based development board. Candidate boards include the Digilent Zybo Z7-20 ($280), Avnet MiniZed, or similar Zynq-7020 platforms.

2.2 Zynq-7020 SoC Specifications

Processing System (PS) — ARM Side:

- Dual-core ARM Cortex-A9 at 667 MHz
- 32KB L1 instruction cache per core
- 32KB L1 data cache per core
- 512KB shared L2 cache
- Memory controller: DDR3/DDR3L
- Built-in peripherals: GEM Ethernet, SDIO, USB, UART

Programmable Logic (PL) — FPGA Side:

- 85,000 logic cells (53,200 LUTs, 106,400 flip-flops)
- 140 Block RAM tiles (36Kb each = 4.9 Mbit total = 630 KB)
- 220 DSP48E1 slices (single-cycle 32-bit integer multiply)
- Operating frequency: 150 MHz target for custom logic

Interconnect:

- 4 AXI HP (High Performance) ports: 64-bit each, direct to DDR3 controller
- 2 AXI GP (General Purpose) ports: 32-bit, for control registers
- DDR3 sustained bandwidth: ~2 GB/s

2.3 Board-Level Resources

- 1 GB DDR3 RAM
- MicroSD card slot (boot and storage)
- HDMI output
- USB OTG (keyboard, mouse)
- Gigabit Ethernet
- UART over USB (serial debug)

---

3. System Architecture

3.1 Execution Model

Entity data loads to the FPGA once at scene start. Each frame, the ARM signals "go." The FPGA runs the complete Silo pipeline internally — fact generation, state machine evaluation, Prolog filtering, UtilityAI scoring, logic block execution, envelope processing, trace recording — all inside core BRAMs and registers. No data returns to DDR3 between passes. When the pipeline completes, the ARM reads back only the render-relevant fields (position, sprite, animation state) for framebuffer output.

Scene load (once):

ARM → DDR3 → FPGA DMA → Core BRAMs: all entity data

Per frame:

ARM: write CONTROL = 1

FPGA (internal, no DDR3 traffic):

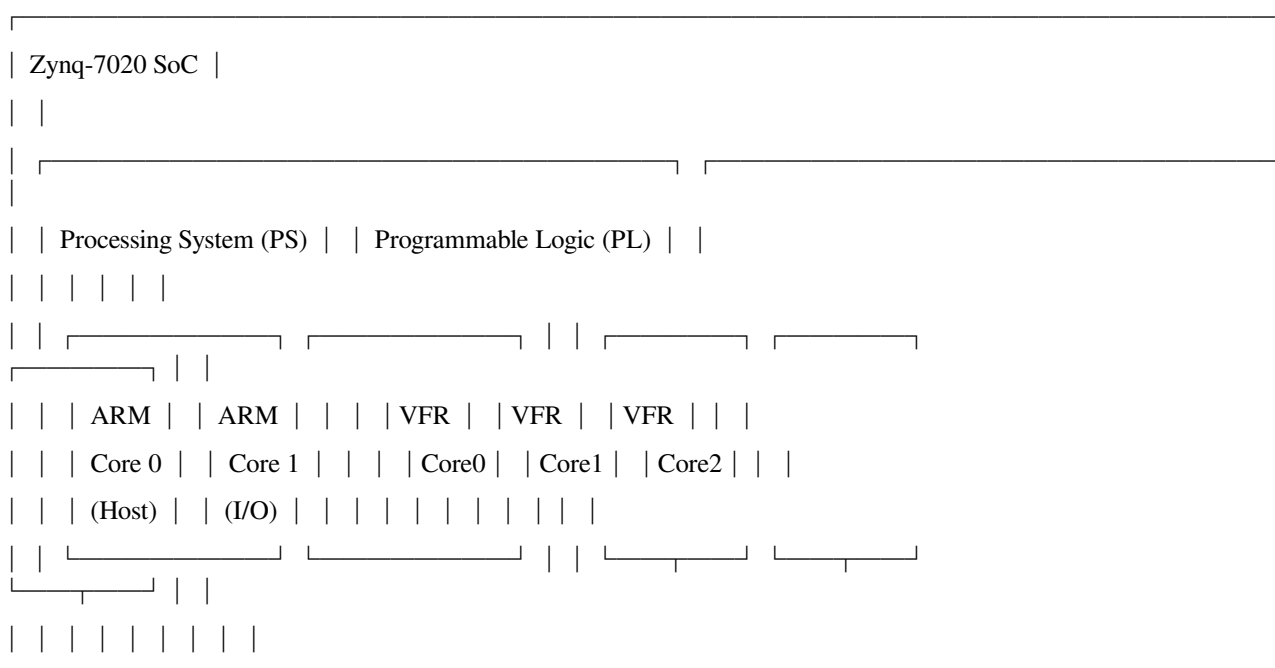
Pass 1-7: all Silo pipeline passes, data in registers and BRAM

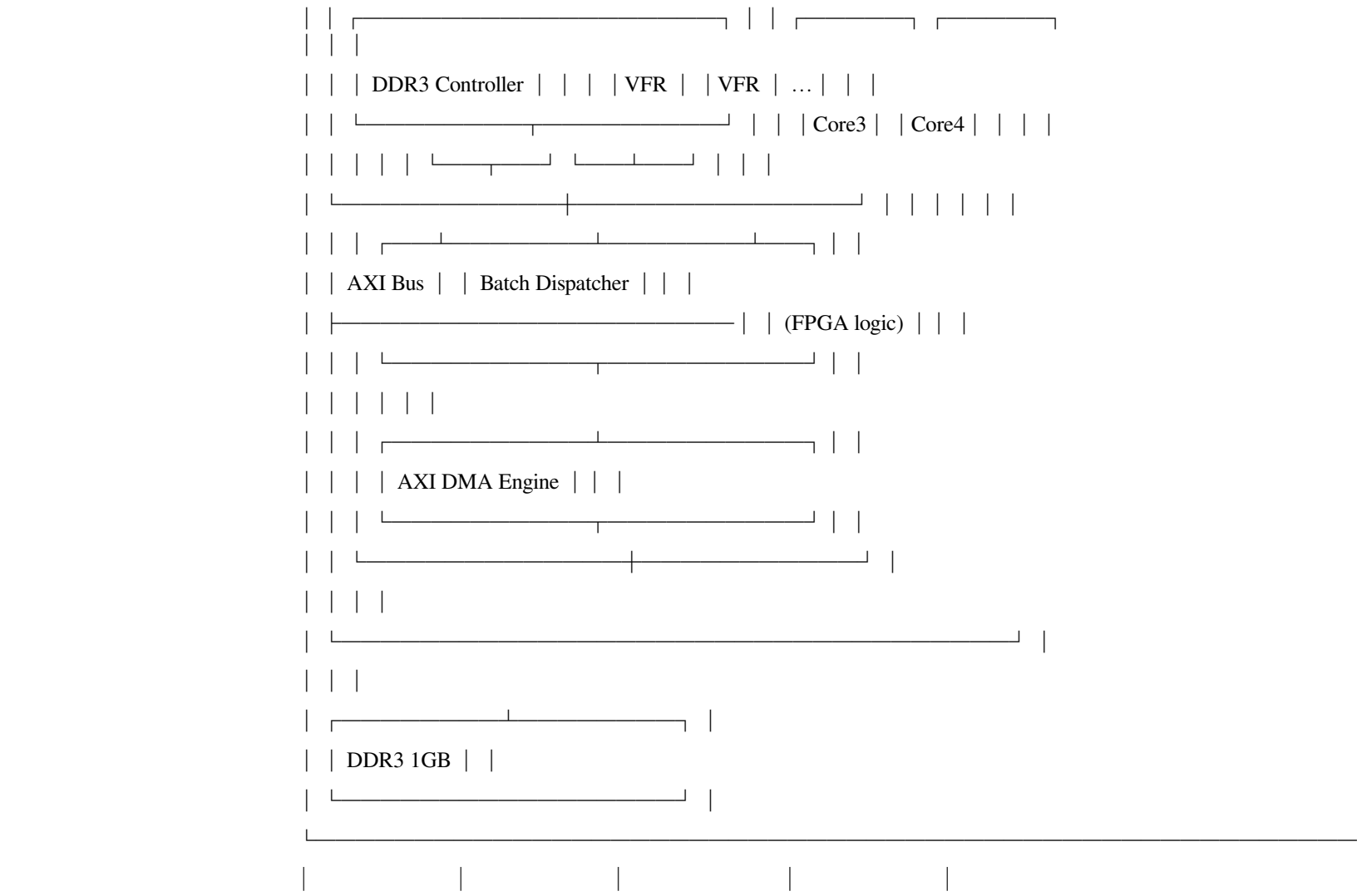
FPGA: write STATUS = done

ARM: DMA read render fields only (5 fields × 5 bytes per entity)

ARM: render framebuffer, handle I/O

3.2 Block Diagram





---

4. VFR Core FPGA Implementation

4.1 Single Core Resource Estimate

Component LUTs Flip-Flops Block RAM DSP48

Instruction fetch 100 64 0 0

Instruction decode 200 128 0 0

ALU (32-bit) 400 256 0 1

- Adder/Subtractor 100 64
- Barrel shifter 150 96
- Bitwise ops 50 32
- Comparator 100 64
- Multiplier 0 0 1 (DSP48)

Register file 250 576 0 0

- V0-V15 (16 × 32b) 128 512
- R0-R15 (16 × 8b) 64 64
- Batch control 58 -

Batch control regs 80 112 0 0

Memory interface 150 96 0 0

DMA interface 200 128 0 0

Total per core ~1,400 ~1,400 0 1

Plus local memory: 2 BRAM tiles (9 KB)

4.2 Core Count

Available: 53,200 LUTs, 106,400 FFs, 140 BRAM, 220 DSP48

Overhead: 5,000 LUTs (dispatcher + AXI DMA + interconnect)

Per core: 1,400 LUTs, 1,400 FFs, 2 BRAM, 1 DSP48

Cores from LUTs: (53,200 - 5,000) / 1,400 = 34

Cores from FFs: 106,400 / 1,400 = 76

Cores from BRAM: 140 / 2 = 70

Cores from DSP48: 220 / 1 = 220

Limiting factor: LUTs

Practical target: 30 VFR cores at 150 MHz

4.3 Zero-Scratch Memory Layout

Entity data writes back into itself. Registers are the working space. There is no separate scratch region. Each core's 9 KB BRAM holds only entity data.

Entity at depth 48: 48 pairs × 5 bytes = 240 bytes per entity

9,216 bytes per core / 240 bytes = 38 entities per core

38 entities × 30 cores = 1,140 entities per chunk

Core BRAM layout:

Core N local BRAM (9,216 bytes):

0x0000 - 0x0FFF Entity data buffer A (38 entities × 240 bytes = 9,120 bytes)

0x23A0 - 0x23FF Core instruction cache (96 bytes, current operation)

For double buffering (load next chunk while processing current):

Core N local BRAM (9,216 bytes):

Buffer A: 0x0000 - 0x11FF 19 entities × 240 = 4,560 bytes (processing)

Buffer B: 0x1200 - 0x23FF 19 entities × 240 = 4,560 bytes (DMA loading)

19 entities × 30 cores = 570 entities per chunk

4.4 Fused Pipeline Execution

All 7 Silo pipeline passes run as one fused operation per entity inside the core. Entity fields load into V registers once, all passes execute in registers, results write back to BRAM once. No intermediate BRAM access between passes.

Fused pipeline per entity (cycle estimate):

Load entity fields into V0-V15: 20 cycles

Pass 1 - Fact generation (register ops): 15 cycles

Pass 2 - State machine eval (CMP, JEQ): 20 cycles

Pass 3 - Prolog filtering (CMP chain): 25 cycles

Pass 4 - UtilityAI scoring (MUL, SHR): 35 cycles

Pass 5 - Logic block execution (ALU ops): 25 cycles

Pass 6 - Envelope processing (SUB, MUL): 15 cycles

Pass 7 - Trace recording (STV): 10 cycles

Write modified fields back to BRAM: 10 cycles

Total per entity: 165 cycles

At 150 MHz: 165 / 150,000,000 = 1.1 µs per entity per core

The 16 V registers hold the entity's hot fields across all passes. The 16 R registers hold remainders. The batch control registers (BF, BC, BD, BI) track position. No register spill. No BRAM read between passes.

---

5. Performance Analysis

5.1 Compute Throughput

Per entity per core: 1.1 µs (165 cycles at 150 MHz)

30 cores parallel: 0.037 µs effective per entity

5.2 DMA Bandwidth

DDR3 sustained bandwidth: ~2 GB/s across all AXI HP ports.

Per-chunk DMA (single buffer, 38 entities per core):

Load: 1,140 entities × 240 bytes = 273,600 bytes

Store: 1,140 entities × 25 bytes = 28,500 bytes (render fields only)

Total: 302,100 bytes per chunk

Transfer time: 302,100 / 2,000,000,000 = 0.15 ms

Per-chunk compute:

1,140 entities / 30 cores = 38 entities per core

38 × 1.1 µs = 41.8 µs = 0.042 ms

DMA dominates. Compute is 3.6x faster than data transfer.

5.3 Entity Count at 60fps — Single Buffer

No double buffering. Load chunk, process, store render results, repeat.

Per chunk: 0.15 ms (DMA) + 0.042 ms (compute) + 0.015 ms (render readback)

```
 = 0.207 ms per 1,140 entities
```

Frame budget: 16.67 ms

Chunks per frame: 16.67 / 0.207 = 80.5

Total entities: 80.5 × 1,140 = 91,800 entities at 60fps

5.4 Entity Count at 60fps — Double Buffer

Load chunk N+1 while processing chunk N. Effective time is the slower of DMA or compute.

Per chunk: max(DMA, compute) = max(0.15, 0.042) = 0.15 ms

Plus render readback: 0.007 ms (570 entities × 25 bytes)

Effective per chunk: 0.157 ms per 570 entities (double buffer = 19 per core)

Chunks per frame: 16.67 / 0.157 = 106

Total entities: 106 × 570 = 60,400 entities at 60fps

Double buffering loses entity capacity per chunk (19 vs 38 per core) but hides DMA latency. Single buffer wins at this scale because compute is so fast relative to DMA.

5.5 Entity Count at 60fps — DDR3 Bandwidth Ceiling

At large entity counts, total data transfer per frame hits the DDR3 bandwidth wall.

Per entity per frame:

Load: 240 bytes (full entity, once)

Store: 25 bytes (render fields, once)

Total: 265 bytes per entity per frame

DDR3 bandwidth: 2 GB/s

Frame budget: 16.67 ms

Max bytes per frame: 2,000,000,000 × 0.01667 = 33,340,000 bytes

Max entities from bandwidth: 33,340,000 / 265 = 125,800 entities

Compute check at 125,800 entities:

125,800 entities / 30 cores = 4,193 entities per core

4,193 × 1.1 µs = 4.61 ms

4.61 ms compute fits in 16.67 ms budget. DDR3 bandwidth is the hard ceiling.

5.6 Practical Ceiling

Accounting for AXI arbitration overhead, DDR3 refresh cycles, ARM render read contention, and dispatcher management cycles at roughly 80% efficiency:

Practical ceiling: 125,800 × 0.80 = ~100,000 entities at 60fps

5.7 Summary Table

Configuration Entities at 60fps

———————————— ————————————————

ARM only (no FPGA, software baseline) 1,900

FPGA single buffer, fused pipeline 91,800

FPGA double buffer, fused pipeline 60,400

DDR3 bandwidth theoretical ceiling 125,800

Practical ceiling (80% efficiency) ~100,000

Single buffer with fused pipeline is the optimal configuration. The compute is fast enough that DMA stalls are brief and double buffering's reduced capacity per chunk is not worth the complexity.

5.8 Comparison to x86 Silo Benchmark

Platform Entities at 60fps Cost Power

———————————— ———————————————— ———————— ————————

x86 Ryzen 16-core 10,000 ~$1,500 125W

Zynq-7020 ARM+FPGA ~100,000 $343 5W

10x more entities. 4x less cost. 25x less power.

———————————————————

6. Memory Map

6.1 DDR3 Layout (ARM View)

0x0000_0000 - 0x001F_FFFF ARM boot and kernel code (2 MB)

0x0020_0000 - 0x00FF_FFFF Silo OS heap and stack (14 MB)

0x0100_0000 - 0x01FF_FFFF Framebuffer (16 MB, double buffered)

0x0200_0000 - 0x1FFF_FFFF Entity data (480 MB)

```
100,000 entities $\times$ 240 bytes = 24 MB active

Remainder: scene data, behavior sets, state machines
```

0x2000_0000 - 0x27FF_FFFF Fact batch storage (128 MB)

0x2800_0000 - 0x2FFF_FFFF Trace storage (128 MB)

0x3000_0000 - 0x37FF_FFFF Network and I/O buffers (128 MB)

0x3800_0000 - 0x3FFF_FFFF FPGA staging / result area (128 MB)

Total: 1 GB

6.2 FPGA Register Map (ARM View via AXI GP)

0x4000_0000 CONTROL write 1 = start frame, 0 = reset

0x4000_0004 STATUS bit 0: busy, bit 1: done, bit 2: error

0x4000_0008 ENTITY_ADDR DDR3 address of entity batch

0x4000_000C ENTITY_COUNT number of entities to process

0x4000_0010 ENTITY_DEPTH fields per entity (48)

0x4000_0014 RENDER_ADDR DDR3 address for render result output

0x4000_0018 RENDER_FIELDS number of fields to write back (5)

0x4000_001C RENDER_OFFSETS DDR3 address of field offset table

0x4000_0020 SM_ADDR DDR3 address of state machine batches

0x4000_0024 BEHAVIOR_ADDR DDR3 address of behavior set batches

0x4000_0028 FACT_SCHEMA_ADDR DDR3 address of fact generation rules

0x4000_002C ENVELOPE_ADDR DDR3 address of envelope batch

0x4000_0030 CORE_COUNT number of active VFR cores (read-only)

0x4000_0034 CORE_STATUS per-core busy/done bits (read-only)

0x4000_0038 CYCLE_COUNT total cycles for last frame (profiling)

0x4000_003C CHUNK_COUNT chunks processed in last frame (profiling)

The ARM writes all addresses and parameters, then writes CONTROL=1. The FPGA batch dispatcher reads entity data from DDR3 in chunks, distributes to cores, processes the full fused pipeline, writes render results to RENDER_ADDR, and sets STATUS=done.

6.3 VFR Core Local Memory

Core N BRAM (9,216 bytes):

0x0000 - 0x23BF Entity data: 38 entities $\times$ 240 bytes = 9,120 bytes

0x23C0 - 0x23FF Instruction micro-cache: 64 bytes (16 instructions)

No scratch region. Entity fields are read into registers, modified in registers, written back to the same BRAM locations. The instruction micro-cache holds the current pipeline stage's instructions, loaded by the dispatcher at scene start.

---

7. Batch Dispatcher (FPGA Logic)

7.1 Frame Processing Flow

1. ARM writes entity address, count, depth, and all batch addresses to registers
2. ARM writes CONTROL = 1
3. Dispatcher computes:

   entities_per_core = 38

   entities_per_chunk = 38 × 30 = 1,140

   total_chunks = ceil(entity_count / 1,140)
4. For each chunk:
   a. DMA load: read 1,140 entities from DDR3 into core BRAMs

Core 0 gets entities [0..37]

Core 1 gets entities [38..75]

…

Core 29 gets entities [1102..1139]

   b. Signal all cores: start

   c. Each core runs fused pipeline on its 38 entities:

For each entity:

```
Load fields into V registers (20 cycles)

Fact generation (15 cycles)

State machine eval (20 cycles)

Prolog filtering (25 cycles)

UtilityAI scoring (35 cycles)

Logic block execution (25 cycles)

Envelope processing (15 cycles)

Trace recording (10 cycles)

Write back to BRAM (10 cycles)
```

   d. All cores signal done

   e. DMA store: write render fields from core BRAMs to DDR3

Only 5 fields × 5 bytes = 25 bytes per entity

1,140 × 25 = 28,500 bytes

f. Advance to next chunk

5. All chunks complete. Set STATUS = done.

7.2 Shared Read-Only Data

State machines, behavior sets, fact schemas, and envelope batches are read-only during frame processing. The dispatcher loads these once at scene start into a shared BRAM region accessible by all cores, or each core caches its needed subset.

Shared data loaded once:

State machine batches: ~2 KB

Behavior set batches: ~10 KB

Fact generation rules: ~4 KB

Curve lookup tables: ~8 KB (256 entries × 4 bytes × 8 curve types)

Total: ~24 KB

Allocated from BRAM pool: 3 BRAM tiles shared across all cores

Remaining for cores: 137 BRAM tiles / 2 per core = 68 cores max (not limiting)

7.3 Resource Estimate

Batch dispatcher: ~2,000 LUTs

AXI DMA engine: ~2,000 LUTs

Shared data BRAM ctrl: ~500 LUTs

Interconnect: ~500 LUTs

Overhead total: ~5,000 LUTs

---

8. ARM Software — Silo OS on Zynq

8.1 Boot Sequence

Power on

→ Zynq boot ROM (silicon)

→ FSBL from SD card (configures DDR3, loads FPGA bitstream)

→ Silo OS kernel from SD card (Zig bare-metal ARM)

→ 8-stage kernel init

→ Main loop

8.2 Zig Bare-Metal Target

zig

// build.zig for Zynq ARM bare-metal

const kernel = b.addExecutable(.{

.name = “silo_kernel”,

.root_module = b.createModule(.{

```
.root_source_file = b.path("src/kernel.zig"),

.target = b.resolveTargetQuery(.{

    .cpu_arch = .arm,

   .cpu_model = .{ .explicit = &std.Target.arm.cpu.cortex_a9 },

    .os_tag = .freestanding,

    .abi = .eabi,

}),

.optimize = .ReleaseSafe,
```

}),

});

8.3 Kernel Init Stages

Stage 1 — CPU:

ARM GIC interrupt controller

Exception vectors

Enable interrupts

Stage 2 — Memory:

DDR3 initialized by FSBL

ARM MMU page tables (identity map 1GB)

Kernel heap

Stage 3 — Hardware:

Read FPGA core count from status register

Verify FPGA responds

Load shared data (state machines, behaviors, curves) to FPGA

Stage 4 — Storage:

SDIO controller (Zynq built-in)

FAT32 filesystem on SD card

Load scene data

Stage 5 — Peripherals:

USB-HID (keyboard, mouse)

HDMI framebuffer

Stage 6 — Network:

GEM Ethernet controller (Zynq built-in)

DHCP, TCP/IP stack

Stage 7 — FPGA Link:

Load entity data from SD card to DDR3

DMA entity data to FPGA core BRAMs

Run test frame, verify results

Measure round-trip latency

Calibrate chunk sizes

Stage 8 — Main Loop:

Enter Silo frame loop

8.4 Driver Mapping

x86-64 Silo OS Zynq ARM Silo OS

———————————— ————————————————

idt.zig (x86 IDT) → gic.zig (ARM GIC)

apic.zig (x86 APIC) → gic.zig (ARM GIC)

port_io.s (x86 in/out) → removed (ARM memory-mapped I/O native)

isr_stubs.s (x86 ISRs) → exception_vectors.s (ARM)

ahci.zig / nvme.zig → sdio.zig (SD card)

nic.zig (E1000) → gem.zig (Zynq Ethernet)

ps2.zig → usb_hid.zig (USB keyboard/mouse)

graphics.zig → unchanged (framebuffer is framebuffer)

context_switch.s → arm_context_switch.s (ARM registers)

Network stack layers unchanged: arp.zig, ip.zig, tcp.zig, udp.zig, dhcp.zig, dns.zig, http.zig. They operate on packet data, not hardware.

8.5 Frame Loop

zig

```
fn runFrame(scene: *Scene) void {
// — Dispatch to FPGA —
const regs = [?](*FPGARegs, [?](0x4000_0000));
regs.entity_addr.* = scene.entity_ddr3_addr;
regs.entity_count.* = scene.actor_count;
regs.entity_depth.* = 48;
regs.render_addr.* = scene.render_ddr3_addr;
regs.render_fields.* = 5;
regs.sm_addr.* = scene.sm_ddr3_addr;
regs.behavior_addr.* = scene.behavior_ddr3_addr;
regs.envelope_addr.* = scene.envelope_ddr3_addr;
// Start FPGA processing
regs.control.* = 1;
// — ARM does I/O while FPGA works —
processInput(scene);
processNetwork(scene);
// — Wait for FPGA —
while (regs.status.* & 0x02 == 0) {}
// — Read render results —
// FPGA has written position + visual data to render_addr
// ARM reads 5 fields × 5 bytes × entity_count
renderFrame(scene, scene.render_ddr3_addr);
```

// — Profiling —

scene.last_cycle_count = regs.cycle_count.*;

scene.last_chunk_count = regs.chunk_count.*;

}

The ARM processes input and network packets in parallel with FPGA computation. When the FPGA signals done, the ARM reads the render results and draws the frame.

---

9. FPGA Design Files

9.1 Module List

fpga_top.v Top-level: N cores + dispatcher + AXI bridge

vfr_core.v Single VFR core (fetch + decode + ALU + registers + BRAM)

vfr_alu.v ALU: shift, add, sub, mul, bitwise, compare

vfr_decode.v Instruction decoder (6-bit opcode → control signals)

vfr_registers.v Register file: V0-V15, R0-R15, batch control, flags

vfr_bram_if.v Local BRAM read/write interface

batch_dispatcher.v Chunk management, core signaling, DMA coordination

axi_dma_bridge.v AXI HP interface for DDR3 transfers

shared_bram.v Shared read-only data (state machines, behaviors, curves)

9.2 Top-Level Instantiation

verilog

module fpga_top #(

parameter NUM_CORES = 30,

parameter BRAM_DEPTH = 2304, // 9,216 bytes in 32-bit words

parameter ENTITY_DEPTH = 48,

parameter ENTITY_BYTES = 240

)(

input wire axi_aclk,

input wire axi_aresetn,

// AXI GP slave (ARM control registers)

// AXI HP master (DDR3 DMA)

```
);
reg [31:0] control_reg;
reg [31:0] status_reg;
reg [31:0] entity_addr_reg;
reg [31:0] entity_count_reg;
reg [31:0] render_addr_reg;
wire [NUM_CORES-1:0] core_start;
wire [NUM_CORES-1:0] core_done;
genvar i;
generate
for (i = 0; i < NUM_CORES; i = i + 1) begin : cores
    vfr_core #(
        .CORE_ID(i),
        .BRAM_DEPTH(BRAM_DEPTH),
        .ENTITIES_PER_CORE(38)
    ) core_inst (
        .clk(axi_aclk),
        .reset(~axi_aresetn),
        .start(core_start[i]),
        .done(core_done[i]),
        .bram_addr(bram_addr[i]),
        .bram_wdata(bram_wdata[i]),
        .bram_rdata(bram_rdata[i]),
        .bram_we(bram_we[i]),
        .shared_bram_addr(shared_addr[i]),
        .shared_bram_rdata(shared_rdata)
```

```
    );
end
endgenerate
batch_dispatcher #(
.NUM_CORES(NUM_CORES),
.ENTITIES_PER_CORE(38)
) dispatcher (
.clk(axi_aclk),
.reset(~axi_aresetn),
.control(control_reg),
.status(status_reg),
.entity_addr(entity_addr_reg),
.entity_count(entity_count_reg),
.render_addr(render_addr_reg),
.core_start(core_start),
.core_done(core_done),
// AXI HP master signals
.m_axi_araddr(m_axi_araddr),
.m_axi_rdata(m_axi_rdata),
.m_axi_awaddr(m_axi_awaddr),
.m_axi_wdata(m_axi_wdata)
);
shared_bram shared_data (
.clk(axi_aclk),
.addr(shared_addr_mux),
```

```
.rdata(shared_rdata)
);
endmodule
```

9.3 ALU

verilog

```
module vfr_alu (
input wire [5:0] opcode,
input wire [31:0] operand_a,
input wire [31:0] operand_b,
input wire [17:0] immediate,
output reg [31:0] result,
output reg [7:0] result_r, // remainder result for VFR ops
output reg flag_eq,
output reg flag_lt,
output reg flag_gt,
output reg flag_ov,
output reg flag_done
);
always @(*) begin
flag_eq = 0; flag_lt = 0; flag_gt = 0; flag_ov = 0;
result_r = 8'd0;
case (opcode)
    // Arithmetic
   6'h08: result = operand_a + operand_b;            // ADD
   6'h09: result = operand_a - operand_b;            // SUB
   6'h0A: result = operand_a * operand_b;            // MUL (DSP48)
   6'h0B: result_r = operand_a[7:0] + operand_b[7:0];    // ADDR
```

```
   6'h0C: result_r = operand_a[7:0] - operand_b[7:0];    // SUBR
    // Bitwise
   6'h10: result = operand_a & operand_b;             // AND
   6'h11: result = operand_a | operand_b;             // OR
   6'h12: result = operand_a ^ operand_b;             // XOR
   6'h13: result = ~operand_a;                      // NOT
    // Shift
   6'h18: result = operand_a >> immediate;            // SHR
   6'h19: result = operand_a << immediate;            // SHL
   6'h1A: result = operand_a >> 5;                  // SHR5
   6'h1B: result = operand_a << 5;                  // SHL5
    // VFR
   6'h20: begin                                   // DECOMP
        result   = operand_a >> 5;
        result_r = operand_a[4:0];
    end
   6'h21: result = (operand_a << 5) | (operand_b[4:0]);  // RECOMP
   6'h22: begin                                    // RNORM
        result_r = operand_a[7:0];
     if ($signed(operand_a[7:0]) > 31)  result_r = operand_a[7:0] -
32;
     if ($signed(operand_a[7:0]) < -32) result_r = operand_a[7:0] + 32;
    end
```

```
   6'h23: begin                                    // RACCUM
        result_r = operand_a[7:0] + operand_b[7:0];
      flag_ov = ($signed(result_r) > 127) | ($signed(result_r) < -
128);
     end
   6'h24: result = operand_a << (immediate * 5);        // SCALE
     // Compare
   6'h28: begin                                      // CMP
        flag_eq = (operand_a == operand_b);
      flag_lt = ($signed(operand_a) < $signed(operand_b));
      flag_gt = ($signed(operand_a) > $signed(operand_b));
     end
   6'h29: begin                                     // CMPR
        flag_eq = (operand_a[7:0] == operand_b[7:0]);
      flag_lt = ($signed(operand_a[7:0]) < $signed(operand_b[7:0]));
      flag_gt = ($signed(operand_a[7:0]) > $signed(operand_b[7:0]));
     end
     default: result = 32'd0;
endcase
end
endmodule
```

---

10. Development Workflow

10.1 Phase 1 — ARM Only (Week 1-4)

Port Silo OS to ARM Cortex-A9 bare-metal. Boot on Zynq. Serial console, HDMI framebuffer, USB input, SD card filesystem, network stack. Run complete Silo system in software on ARM. All entity processing on ARM, no FPGA. Target: 1,900 entities at 60fps.

Validates: Silo OS runs on Zynq. All I/O works. Software baseline established.

10.2 Phase 2 — Single VFR Core (Week 5-8)

Design vfr_core.v. Simulate all 35 instructions in Verilator. Synthesize single core. ARM writes test batch to DDR3, triggers FPGA, single core processes, ARM reads result. Verify against software.

Validates: ISA executes correctly in hardware. AXI communication works.

10.3 Phase 3 — Multi-Core (Week 9-12)

Instantiate 4 cores. Implement batch dispatcher. Test chunked processing. Scale to 16, then 30 cores. Benchmark throughput. Profile AXI DMA bandwidth.

Validates: Parallel batch processing works. Linear scaling confirmed.

10.4 Phase 4 — Full Integration (Week 13-16)

Connect frame loop. ARM dispatches, FPGA processes all passes with fused pipeline, ARM renders. Measure entity counts at 60fps. Profile chunk timing, DMA bandwidth, compute utilization. Compare against ARM-only baseline.

Validates: Complete system at target performance.

10.5 Phase 5 — Optimization (Week 17-20)

Tune fused pipeline cycle counts. Optimize DMA patterns. Test selective field readback. Profile DDR3 bandwidth utilization. Push toward 100,000 entity ceiling.

---

11. Scaling Path

11.1 Larger FPGA

Same Verilog, change core count parameter:

Zynq-7020: 53K LUTs → 30 cores → ~100,000 entities $343

Zynq-7045: 218K LUTs → 130 cores → ~350,000 entities $600

Zynq-7100: 444K LUTs → 270 cores → ~700,000 entities $1,500

UltraScale+: 600K LUTs → 400 cores → 1,000,000 entities $3,000

At 400 cores, compute time for 1,000,000 entities: 1,000,000 / 400 × 1.1 µs = 2.75 ms. DDR3 bandwidth for 1M entities at 265 bytes each: 265 MB per frame. Requires DDR4 or multi-channel memory. UltraScale+ boards provide this.

11.2 Multi-Board Cluster

Multiple Zynq boards connected via Gigabit Ethernet:

Board 0 (master): ARM host + 30 VFR cores + HDMI/USB/network

Board 1-N (workers): ARM relay + 30 VFR cores each

Master ARM partitions entities across boards.

Worker ARM receives entity chunks via Ethernet, dispatches to local FPGA.

Worker ARM sends render results back to master via Ethernet.

4 boards: 120 cores, ~350,000 entities, ~$1,400

8 boards: 240 cores, ~650,000 entities, ~$2,800

11.3 Custom ASIC

FPGA-validated Verilog goes to foundry. VFR core at 130nm process: ~0.05 mm^2 per core. 1000 cores on a 10mm × 10mm die. Feasible at older, cheaper process nodes. The FPGA phase produces production-ready RTL.

---

12. Bill of Materials

12.1 Development Setup

Digilent Zybo Z7-20 (Zynq-7020) $280

MicroSD card 32 GB $10

USB keyboard $15

USB mouse $10

HDMI cable $8

5V 2.5A power supply $15

USB Micro-B cable (debug UART) $5

Total: $343

12.2 Development Tools (Free)

Xilinx Vivado WebPACK Edition free (supports Zynq-7020)

Zig 0.14 compiler free

Verilator (Verilog simulation) free, open source

GTKWave (waveform viewer) free, open source

minicom (serial terminal) free

12.3 Optional

Logic analyzer (Saleae clone) $15

Digilent JTAG-HS3 (faster FPGA programming) $60

Second Zybo Z7-20 (cluster experiments) $280

---

13. Power and Thermal

Zynq-7020 SoC total: ~2.5 W

ARM PS: ~1.0 W

FPGA PL (30 cores): ~1.5 W

DDR3 RAM: ~1.0 W

Board peripherals: ~1.5 W

Total system: ~5 W

No heatsink required at this power level. Passive cooling sufficient. Compare: x86 desktop running equivalent Silo workload draws 125-250W.

---

14. Summary

Property	Specification
Platform	Xilinx Zynq-7020 SoC
ARM processor	Dual Cortex-A9 at 667 MHz
FPGA fabric	53,200 LUTs, 140 BRAM, 220 DSP48
VFR cores	30 at 150 MHz
Local memory per core	9 KB Block RAM (38 entities)
Shared read-only BRAM	24 KB (state machines, behaviors, curves)
System memory	1 GB DDR3 (~2 GB/s bandwidth)

Property	Specification
Execution model	Fused 7-pass pipeline, data resident on FPGA
Scratch memory	Zero (registers are working space, writes back to entity)
Entities per chunk	1,140 (38 per core $\times$ 30 cores)
Cycles per entity	165 (fused pipeline)
Entity throughput	~100,000 at 60fps
ARM-FPGA protocol	Write addresses + go signal, poll done, read render results
Display	HDMI via ARM framebuffer
Input	USB keyboard and mouse
Network	Gigabit Ethernet
Storage	MicroSD FAT32
ARM software	Silo OS in Zig bare-metal
FPGA design	VFR cores in Verilog
Development cost	$343
Power consumption	~5 W
Scaling path	Larger FPGA $\rightarrow$ multi-board cluster $\rightarrow$ ASIC

Integer ALU Based Computing — Motherboard Specification v2.0

Companion document to the ISA v1.0, Compiler v1.0, and Silo-VFR Mapping v1.0

[**CKS-0-2026**] Cymatic K-Space Mechanics. Github: https://github.com/ghowland/cks/tree/main/papers/_CKS/CKS-0-2026

[**CKS-MATH-0-2026**] Cymatic K-Space Mechanics. Github: https://github.com/ghowland/cks/tree/main/papers/MATH/C MATH-0-2026

[**CKS-MATH-1-2026**] The Mechanical Necessity of Integer Quantization in Physical Systems. Github: https://github.com/ghowland/cks/tree/main/papers/MATH/CKS-MATH-1-2026

[**CKS-MATH-10-2026**] Grand Unification. Github: https://github.com/ghowland/cks/tree/main/papers/MATH/CKS-MATH-10-2026

[**CKS-MATH-104-2026**] Grand Unification v23. Github: https://github.com/ghowland/cks/tree/main/papers/MATH/CKS MATH-104-2026

[**CKS-LEX-12-2026**] Measurement Systems in the $\mathbb{Q}$-Substrate. Github: https://github.com/ghowland/cks/tree/main/paper LEX-12-2026